

Mi Tutora — Firestore Database Architecture

Date: 2026-06-17 Firebase Project: tutor-app-1e394

Table of Contents

1. [Collections Overview](#)
 2. [Document Schemas](#)
 3. [Authentication & User Relationships](#)
 4. [Entity Relationships \(ER Diagram\)](#)
 5. [Source of Truth Analysis](#)
 6. [Denormalized & Duplicated Fields](#)
 7. [Application Lifecycle](#)
 8. [Student Lifecycle](#)
 9. [Tutor Lifecycle](#)
 10. [Payment Lifecycle](#)
 11. [Global Config Architecture](#)
 12. [Cloud Functions](#)
 13. [Recommended Indexes](#)
 14. [Data Consistency Risks](#)
-

1. Collections Overview

Collection	Document ID Strategy	Approximate Role
users	Firebase Auth UID	Auth metadata + role + FCM token for ALL users
parents	Firebase Auth UID	Parent profile (area, city, phone)
tutors	Firebase Auth UID	Full tutor profile (category, subjects, pricing, location)
students	Auto-generated	Student linked to a parent; one parent may have many
tuition_requests	Auto-generated	Parent's open marketplace posting
tutor_requests	Auto-generated	Tutor's proactive expression of interest in a student profile

Collection	Document ID Strategy	Approximate Role
applications	Auto-generated or composite	Central joining doc: tutor ↔ request/student; full deal lifecycle
direct_requests	Auto-generated or composite	Parent-initiated direct hire of a specific tutor (separate flow)
referrals	UUID string	Tracks who referred whom and reward status
global_config	Singleton "app"	Remote kill-switch, maintenance mode, version gating

No subcollections are used. All collections are at Firestore root level.

2. Document Schemas

2.1 users/{uid}

Identity and notification routing for every authenticated user (both roles).

Field	Type	Default	Notes
id	String	—	Mirrors document ID (Firebase UID)
name	String	—	Display name
email	String	—	From Auth; Google or email/password
role	String	""	"parent" or "tutor"; empty until role selection
hasProfile	Boolean	false	Set true after profile setup completes
referralCode	String	""	User's own shareable referral code
referredBy	String	""	referralCode of the person who referred this user
fcmToken	String	""	Device FCM token; updated on every mainScreen load
freeDemoEligible	Boolean	false	Granted to tutors whose demo slot was taken by another tutor

Written by: `AuthViewModel.signup()`,
`UserRepository.saveGoogleUserIfNew()`, `UserRepository.saveFcmToken()`,
`ParentViewModel.appointTutor()` (freeDemoEligible),
`ReferralViewModel.claimReferralCode()`

2.2 parents/{uid}

Parent profile data. Always keyed by Firebase UID.

Field	Type	Notes
name	String	Display name (synced from users)
area	String	Neighbourhood
city	String	City
phone	String	Contact number; read by tutors for call action
hasStudent	Boolean	Set <code>true</code> when first student is added

Written by: `UserRepository.updateParentProfile()` (batch with users),
`StudentViewModel.addStudent()` (sets `hasStudent: true`)

2.3 tutors/{uid}

Full tutor profile. Always keyed by Firebase UID.

Field	Type	Notes
id	String	Mirrors document ID
category	String	"school", "programming", or "languages"
name	String	Display name
gender	String	
phone	String	
whatsapp	String	
email	String	
address	String	
qualification	String	
experience	String	
occupation	String	

Field	Type	Notes
subjects	List<String>	School tuition only
classes	List<String>	School tuition only (class levels taught)
boards	List<String>	School tuition only
technologies	List<String>	Programming tuition only
languagesTaught	List<String>	Languages tuition only
knownLanguages	List<String>	Communication languages
mode	String	"online", "offline", or "both"
teachingApproach	String	
studentCount	String	Self-reported active student count
schoolNames	String	Schools associated with
preferredLocations	String	
preferredTimeRange	String	
travelDistance	String	Max travel radius
feeRange	String	Display range (e.g. "₹500–₹800/hr")
price	Int	Canonical monthly price (used in recommendations)
rating	Double	Default 0.0; TutorRepository defaults missing values to 4.5
area	String	
city	String	
latitude	Double	
longitude	Double	Used by RecommendationEngine for distance filtering
hasProfile	Boolean	true after profile setup

Read type coercion note: `TutorRepository.getAllTutors()` handles Firestore returning Long for rating and Long/Double for price, converting them to the correct Kotlin types. This is a known Firestore/Kotlin deserialization quirk with numeric fields.

2.4 students/{autoId}

One student per document. A parent may have many students.

Field	Type	Default	Notes
id	String	""	Written as body field (not just document ID)
parentId	String	""	Firestore UID of the owning parent
category	String	""	"school", "programming", "languages"
name	String	""	Student display name
gender	String	""	
phoneNumber	String	""	
whatsappNumber	String	""	
email	String	""	
address	String	""	
studentType	String	""	
classLevel	String	""	School only
board	String	""	School only
subjects	List<String>	[]	School only
technologies	List<String>	[]	Programming only
languages	List<String>	[]	Languages only
budget	Int	0	Monthly budget in ₹
preferredMode	String	""	"online" / "offline"
guardianName	String	""	
hoursPerDay	String	""	
daysPerWeek	String	""	
learningGoal	String	""	
specialRequirements	String	""	
dob	String	""	Date of birth (string format)
createdAt	Long	now()	Epoch milliseconds

2.5 tuition_requests/{autoId}

A parent's public marketplace listing. Tutors browse and apply to these.

Field	Type	Default	Notes
id	String	""	Written as body field
parentId	String	""	Owning parent UID
category	String	""	"school", "programming", "languages"
studentId	String	""	Linked student document ID
studentName	String	""	Denormalized from student
classLevel	String	""	Denormalized from student
board	String	""	Denormalized from student
subjects	List<String>	[]	Denormalized from student
technologies	List<String>	[]	Denormalized from student
languages	List<String>	[]	Denormalized from student
budget	Int	0	Max monthly budget in ₹
mode	String	""	"online" / "offline"
preferredTimeRange	String	""	
latitude	Double	0.0	Denormalized from student/parent
longitude	Double	0.0	Denormalized from student/parent
area	String	""	Denormalized
city	String	""	Denormalized
status	String	"open"	"open" → "closed" when tutor appointed
acceptedTutorId	String	""	Populated when parent appoints a tutor
createdAt	Long	now()	Epoch milliseconds

2.6 tutor_requests/{autoId}

A tutor's proactive interest in a student profile (tutor → parent direction). Separate from the marketplace applications flow.

Field	Type	Default	Notes
id	String	""	
tutorId	String	""	
tutorName	String	""	Denormalized from users
parentId	String	""	Target parent
tuitionRequestId	String	""	Linked tuition_requests doc (if applicable)
studentId	String	""	Target student
studentName	String	""	Denormalized from student
classLevel	String	""	Denormalized
category	String	""	
subjects	List<String>	[]	Denormalized
technologies	List<String>	[]	Denormalized
languages	List<String>	[]	Denormalized
budget	Int	0	
mode	String	""	
preferredTimeRange	String	""	
area	String	""	
city	String	""	
status	String	"pending"	"pending" only (no known transitions in current code)
createdAt	Long	now()	

2.7 applications/{id}

The central transaction document. Every parent-tutor deal lives here regardless of how it started. Documents go through an 8-stage status lifecycle.

Document ID strategies:

- Tutor-initiated post application: auto-generated (in applyToTuition)

- Parent-initiated direct: composite "\${parentId}_\${tutorId}_\${requestId}"
- Tutor-initiated direct (expressInterest): auto-generated

Field	Type	Default	Notes
id	String	""	Written as body field for deserialization
tutorId	String	""	
tutorName	String	""	Denormalized from users
requestId	String	""	Links to tuition_requests; blank for direct flow
parentId	String	""	
studentId	String	""	Populated for direct flow; also forward-filled since fix
studentName	String	""	Denormalized snapshot at creation time
currentOffer	Int	0	Active negotiated price
initialBudget	Int	0	Parent's original budget (immutable reference)
lastUpdatedBy	String	""	"parent" or "tutor" (turn-based negotiation gating)
status	String	"applied"	See lifecycle state machine below
source	String	"post"	"post" (tutor applied) or "direct" (either party initiated direct)
finalPrice	Int	0	Agreed price; set on deal acceptance
demoPaymentPaid	Boolean	false	
demoPaymentLink	String	""	Payment gateway URL
meetingLink	String	""	Online meeting URL (tutor-added)
mode	String	"offline"	Copied from request at creation
meetingNotes	String	""	
demoDate	String	""	
demoTime	String	""	
studentLatitude	Double	0.0	Copied from request at

Field	Type	Default	Notes
			creation
studentLongitude	Double	0.0	Copied from request at creation
studentAddress	String	""	Copied from request at creation
scheduledDemoDate	String	""	Confirmed demo date
scheduledDemoTime	String	""	Confirmed demo time
proposedDemoDate	String	""	Latest counter-proposal
proposedDemoTime	String	""	Latest counter-proposal
demoScheduleStatus	String	""	"proposed" or "accepted"
demoScheduleUpdatedBy	String	""	"parent" or "tutor"
nextPaymentDate	String	""	
tuitionPaymentsStatus	String	"pending"	
tuitionPaymentLink	String	""	
lastPaymentDate	String	""	
meetingPlatform	String	""	e.g. "zoom", "meet"
category	String	""	Copied from request at creation
startDate	String	""	dd/MM/yyyy; set when tuition starts
nextDueDate	String	""	dd/MM/yyyy; set when tuition starts

Status values: applied → negotiating → demo_pending_payment → demo_booked → tuition_started

Terminal statuses: rejected, withdrawn

2.8 direct_requests/{id}

Parent-initiated direct hire. A lighter document than `Application` — no negotiation, just accept/reject.

Field	Type	Default	Notes
id	String	""	
tutorId	String	""	
tutorName	String	""	Denormalized
parentId	String	""	
studentId	String	""	
status	String	"pending"	"pending" / "accepted" / "rejected"
price	Int	0	
createdAt	Long	now()	

Note: `TutorViewModel.acceptDirectRequest()` and `rejectDirectRequest()` update this collection. Creation point is unclear from current codebase — this collection may be partially implemented or maintained alongside the `applications` direct flow. See [§14](#).

2.9 referrals/{uuid}

One document per referral event.

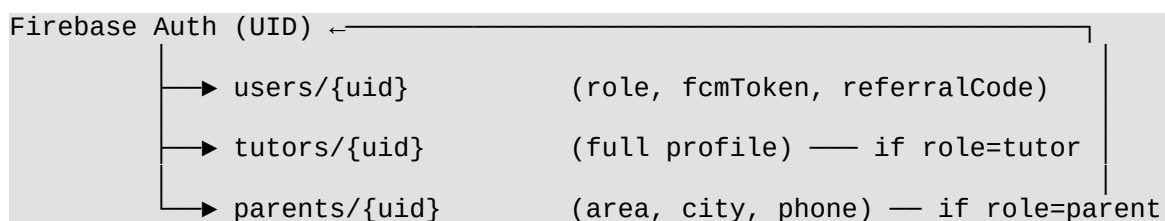
Field	Type	Default	Notes
id	String	""	UUID (not document ID — written as body field)
referrerId	String	""	UID of the person who shared the code
referrerName	String	""	Denormalized
referredUserId	String	""	UID of the new user
referredUserName	String	""	Denormalized
referralCode	String	""	The code that was used
referralType	String	""	"signup"
status	String	"pending"	"pending" / "active" / "rewarded"
estimatedReward	Int	0	₹ reward amount
createdAt	Long	now()	

2.10 global_config/app

Singleton document. Always document ID "app".

Field	Type	Default	Notes
appEnabled	Boolean	true	Kill-switch; false → show disabled screen
maintenanceMode	Boolean	false	Shows maintenance screen with title/message
maintenanceTitle	String	""	
maintenanceMessage	String	""	
disabledTitle	String	""	Shown when appEnabled = false
disabledMessage	String	""	
minSupportedVersionCode	Long	0	Force-upgrade if app's versionCode < this
recommendedVersionCode	Long	0	Soft-upgrade suggestion
latestVersionName	String	""	e.g. "1.3.0"
announcementEnabled	Boolean	false	In-app banner toggle
announcementTitle	String	""	
announcementMessage	String	""	

3. Authentication & User Relationships



Login flows:

Method	New user path	Returning user path
Email/password	AuthViewModel.signup()	login() → reads

Method	New user path	Returning user path
	) → creates users/{uid} with role="" → roleSelection	users/{uid}.role → role-based home
Google Sign-In	UserRepository.saveGoogleUserIfNew() — checks existence; if new, creates users/{uid} with role="" → roleSelection	Document already exists; role read from Firestore → cached in DataStore

Role persistence:

1. Firestore users/{uid}.role — authoritative source
2. DataStoreManager (local Android DataStore) — cache, read on every app launch to avoid a Firestore round-trip

Profile setup guard: users/{uid}.hasProfile — if false after login, user is redirected to profile setup screens. Set to true by profile setup ViewModels.

4. Entity Relationships (ER Diagram)

```
erDiagram
    USERS {
        string id PK
        string role
        string fcmToken
        string referralCode
        string referredBy
        boolean hasProfile
        boolean freeDemoEligible
    }

    PARENTS {
        string uid PK
        string phone
        string area
        string city
    }

    TUTORS {
        string uid PK
        string category
        string name
        int price
        double latitude
        double longitude
    }

    STUDENTS {
        string id PK
        string parentId FK
        string category
    }
```

```

    string name
    int budget
}

TUITION_REQUESTS {
    string id PK
    string parentId FK
    string studentId FK
    string studentName
    string status
    string acceptedTutorId
}

APPLICATIONS {
    string id PK
    string tutorId FK
    string requestId FK
    string parentId FK
    string studentId FK
    string status
    string source
    int finalPrice
}

TUTOR_REQUESTS {
    string id PK
    string tutorId FK
    string parentId FK
    string studentId FK
    string status
}

DIRECT_REQUESTS {
    string id PK
    string tutorId FK
    string parentId FK
    string studentId FK
    string status
}

REFERRALS {
    string id PK
    string referrerId FK
    string referredUserId FK
    string status
}

GLOBAL_CONFIG {
    string id PK
    boolean appEnabled
    boolean maintenanceMode
    long minSupportedVersionCode
}

USERS ||--o| PARENTS : "uid (role=parent)"
USERS ||--o| TUTORS : "uid (role=tutor)"
PARENTS ||--o{ STUDENTS : "parentId"
PARENTS ||--o{ TUITION_REQUESTS : "parentId"
STUDENTS ||--o{ TUITION_REQUESTS : "studentId (denorm)"
TUTORS ||--o{ APPLICATIONS : "tutorId"
TUITION_REQUESTS ||--o{ APPLICATIONS : "requestId"
PARENTS ||--o{ APPLICATIONS : "parentId"
STUDENTS }o--o{ APPLICATIONS : "studentId (denorm)"
TUTORS ||--o{ TUTOR_REQUESTS : "tutorId"

```

```

PARENTS ||--o{ TUTOR_REQUESTS : "parentId"
STUDENTS }o--o{ TUTOR_REQUESTS : "studentId (denorm)"
TUTORS ||--o{ DIRECT_REQUESTS : "tutorId"
PARENTS ||--o{ DIRECT_REQUESTS : "parentId"
STUDENTS }o--o{ DIRECT_REQUESTS : "studentId (denorm)"
USERS ||--o{ REFERRALS : "referrerId"
USERS ||--o{ REFERRALS : "referredUserId"

```

5. Source of Truth Analysis

Data	Source of Truth	Copies Exist In
User's name	users/{uid}.name	tutors/{uid}.name, applications.tutorName, tutor_requests.tutorName, referrals.referrerName/referredUserName
User's role	users/{uid}.role	Android DataStore (cache)
Parent's phone	parents/{uid}.phone	Nowhere (fetched on demand)
Tutor's full profile	tutors/{uid}	TutorProfile (UI-only subset, not persisted)
Student data	students/{autoId}	tuition_requests (studentName, classLevel, board, subjects, technologies, languages, location), applications (studentName, studentId), tutor_requests (studentName + curriculum fields)
Tuition request budget	tuition_requests.budget	applications.initialBudget (snapshot at application time)
Student location	students (via parent address)	tuition_requests (latitude, longitude, area, city), applications (studentLatitude, studentLongitude, studentAddress)
FCM token	users/{uid}.fcmToken	Nowhere — single copy, overwritten on each login
Referral code	users/{uid}.referralCode	referrals.referralCode (snapshot at use time)
Global config	global_config/app	Held in-memory by GlobalConfigViewModel

6. Denormalized & Duplicated Fields

High-duplication fields (written once, copied everywhere)

Field	Written From	Copies In	Staleness Risk
tutorName	users/{uid}.name	applications, tutor_requests, direct_requests	High — if tutor updates name, all historical docs show old name. Intentional: historical records should reflect name at time of transaction.
studentName	students/{id}.name	tuition_requests, applications, tutor_requests	Medium — student name can change. Runtime enrichment in EarningsViewModel and TutorHomeScreen always fetches live name from students.
studentId	students/{id}	tuition_requests, applications (forward-filled since fix), tutor_requests	Historical applications (prefix) have studentId="". requestId path is used as fallback.
category	students/{id}.category	tuition_requests, applications, tutor_requests	Low — category rarely changes after creation.
subjects/technologies/languages	students/{id}	tuition_requests, tutor_requests	Low — curriculum fields rarely change.
Location fields	Parent's address input	tuition_requests, applications	Medium — if parent moves, old requests retain old location.
initialBudget	tuition_requests.budget at apply time	applications.initialBudget	Intentionally frozen — tracks original budget before negotiation.

Denormalization patterns

```
students/{id}
└─> tuition_requests (studentName, classLevel, board, subjects, location)
```

```
↳ applications (studentName, studentId, location, category, mode)
```

The applications collection is the farthest denormalized point. Student data flows down two hops and is never updated in-place after the application is created.

7. Application Lifecycle

7.1 State Machine

```
stateDiagram-v2
    [*] --> applied : Tutor applies to post\n(applyToTuition)\nOR tutor
expresses interest\n(expressInterest)\nOR parent sends
direct\n(sendDirectRequest)

    applied --> negotiating : Counter-offer made\n(parentCounter / tutorCounter)
    applied --> demo_pending_payment : Deal accepted at initial price
    applied --> rejected : Either party rejects
    applied --> withdrawn : Tutor withdraws

    negotiating --> negotiating : Another counter-offer\n(turn-based,
lastUpdatedBy enforced)
    negotiating --> demo_pending_payment : Deal accepted\n(acceptDeal /
tutorAcceptDeal)
    negotiating --> rejected : Either party rejects
    negotiating --> withdrawn : Tutor withdraws

    demo_pending_payment --> demo_booked : Parent marks demo payment
paid\n(markDemoPaymentPaid)

    demo_booked --> tuition_started : Parent appoints tutor\n(appointTutor)
\nSets startDate + nextDueDate

    rejected --> [*]
    withdrawn --> [*]
    tuition_started --> [*]
```

7.2 Source distinction

source value	Who created it	How
"post"	Tutor	Tutor browses tuition_requests, clicks Apply
"direct"	Parent	Parent views tutor profile, sends direct request
"direct"	Tutor	Tutor views student profile, expresses interest

7.3 Side-effects on status transitions

Transition	Side-effects on other documents
applied → demo_pending_payment	applications: all OTHER applications for

Transition	Side-effects on other documents
(source=post)	same requestId are rejected. tuition_requests.acceptedTutorId is set.
demo_booked → tuition_started (appointTutor)	applications: other apps for same request rejected. Rejected tutors get users/{tutorId}.freeDemoEligible = true.tuition_requests.status → "closed".

7.4 Negotiation turn-gating

lastUpdatedBy field enforces turn-based negotiation:

- parentCounter() checks lastUpdatedBy != "parent" before proceeding
- tutorCounter() does NOT explicitly check (observed inconsistency — see §14)

8. Student Lifecycle

flowchart TD

```

A[Parent creates account] --> B[students collection empty]
B --> C[Parent adds student\nStudentViewModel.addStudent]
C --> D[students doc created\nparents.hasStudent = true]
D --> E[Parent posts tuition request\nTuitionViewModel.createTuitionRequest]
E --> F[tuition_requests doc created\nstudentId + denorm fields copied]
F --> G[Tutors discover and apply]
G --> H[applications docs created\nstudentName + studentId copied]
H --> I[Deal progresses to tuition_started]
I --> J[EarningsViewModel + TutorHomeScreen\nresolve live name from students doc]
D --> K[Parent edits student\nStudentViewModel.updateStudent]
K --> L[students doc updated\ntuition_requests NOT updated\napplications NOT updated]
D --> M[Parent deletes student\nStudentViewModel.deleteStudent]
M --> N[students doc deleted\nOrphaned tuition_requests remain\nOrphaned applications remain]

```

Key facts:

- Student edits and deletions do NOT cascade to tuition_requests or applications
- Runtime name enrichment (enrichStudentNames, fetchStudentNameForApp) always fetches the live name from students/{id}, so the display is always current even when the application document has a stale snapshot
- Deleting a student orphans all their linked documents (see §14)

9. Tutor Lifecycle

flowchart TD

```

A[Tutor creates account\nAuthViewModel.signup] --> B[users doc
created\nrole=tutor, hasProfile=false]
B --> C[Tutor selects category\ntutor_category_selection route]
C --> D[Tutor fills profile form\ntutors doc created\nusers.hasProfile =
true]
D --> E[Tutor is discoverable\nTutorRepository.getAllTutors]
D --> F[Tutor browses
tuition_requests\nTutorViewModel.observeAvailableTutions]
F --> G[Tutor applies\nTutorViewModel.applyToTuition\napplications doc
created]
D --> H[Tutor views student profiles\nRecommendation flow]
H --> I[Tutor expresses
interest\nTutorViewModel.expressInterest\napplications doc created
source=direct]
H --> J[Tutor sends request\nTutorViewModel.sendTutorRequest\ntutor_requests
doc created]
G --> K[Application lifecycle\nsee §7]
I --> K
K --> L[tuition_started]
L --> M[Tutor tracked in EarningsViewModel\napplications where
status=tuition_started]

```

10. Payment Lifecycle

There are two payment events in the application lifecycle:

10.1 Demo fee payment

```

demo_pending_payment
  ↓ (application.demoPaymentLink provided by tutor or system)
  ↓ Parent pays demo fee externally
  ↓ Parent marks paid in app
markDemoPaymentPaid()
  → applications.demoPaymentPaid = true
  → applications.status = "demo_booked"

```

10.2 Recurring tuition payment

```

tuition_started
  ↓ application.startDate set (dd/MM/yyyy)
  ↓ application.nextDueDate set (dd/MM/yyyy)
  ↓ application.tuitionPaymentLink = payment URL
  ↓ application.tuitionPaymentStatus = "pending"

```

Every payment cycle:

- tuitionPaymentStatus updated
- lastPaymentDate updated
- nextDueDate updated
- nextPaymentDate updated

10.3 Payment state fields on applications

Field	Meaning
demoPaymentPaid	Boolean; true after demo fee is paid
demoPaymentLink	External URL for demo payment

Field	Meaning
finalPrice	Monthly fee agreed by both parties
startDate	When tuition began (dd/MM/yyyy)
nextDueDate	Next monthly payment due (dd/MM/yyyy) — parsed by <code>EarningsViewModel.parseDateMillis()</code>
tuitionPaymentStatus	"pending" / (other values TBD as feature matures)
tuitionPaymentLink	Recurring payment URL
lastPaymentDate	Most recent payment date
nextPaymentDate	Same as <code>nextDueDate</code> (two fields, same concept — see §14)

10.4 EarningsViewModel aggregation

`EarningsViewModel` computes from all applications where `tutorId == currentUser` AND `status == "tuition_started"`:

- **Monthly contracted income:** `sum(finalPrice)`
- **Active student count:** `count(activeTuitions)`
- **Next payment due:** `min(nextDueDate)` across all active tuitions (parsed from dd/MM/yyyy)

11. Global Config Architecture

Purpose

Remote control of the app without a Play Store release. Evaluated on every cold start before the user sees any content.

Document location

`global_config/app` — singleton document. There is only ever one document in this collection.

Evaluation priority (in `GlobalConfigViewModel`)

1. `appEnabled == false` → show `AppDisabledScreen` (hard block)
2. `maintenanceMode == true` → show `MaintenanceScreen` (hard block)
3. `versionCode < minSupportedVersionCode` → show `ForceUpdateScreen` (hard block)
4. `versionCode < recommendedVersionCode` → show `SoftUpdateDialog` (dismissable)
5. `announcementEnabled == true` → show `AnnouncementBanner` (non-blocking)
6. All clear → normal app flow

Realtime listening

`GlobalConfigRepository.observeConfig()` returns a `ListenerRegistration` and fires on every document change. `GlobalConfigViewModel.startListening()` sets up this listener at app start. Any change to the Firestore document is reflected in the UI within seconds.

Access pattern

```
GlobalConfigViewModel (singleton, app-scoped)
└─ GlobalConfigRepository.observeConfig()
    └─ addSnapshotListener on global_config/app
```

No Android ViewModel scoped per-screen — it is app-level state. Any screen can observe `GlobalConfigViewModel.configState` to react to kill-switch changes.

12. Cloud Functions

Currently one function: `notifyParentOnApplication`

Trigger: `onDocumentCreated("applications/{appId}")`

Logic:

```
new applications document created
└ read applications.parentId
└ fetch users/{parentId}
└ read users.fcmToken
└ send FCM push message:
    title: "New Tutor Application"
    body: "{tutorName} applied to your tuition request"
    data.route: "applicants"
    data.requestId: applications.requestId
```

Note: This function fires for ALL new `applications` documents regardless of source. A direct-flow application (`source="direct"`) also triggers this notification. The `requestId` in the notification payload will be blank for direct applications, so the `applicants` deep-link route may not resolve correctly for the direct flow.

13. Recommended Indexes

Firestore auto-creates single-field indexes. Compound queries require manual composite indexes.

Required composite indexes

Collection	Fields	Query location
applications	tutorId ASC, status ASC	EarningsViewModel.observe(), TutorViewModel.observeMyTutions(),

Collection	Fields	Query location
		<code>TutorViewModel.observeIncomingRequests()</code> , <code>TutorViewModel.observeAppliedRequestIds()</code>
applications	parentId ASC, status ASC	<code>ParentViewModel.getActiveTuition()</code> , <code>ParentViewModel.observeActiveTutions()</code>
applications	tutorId ASC, source ASC	<code>TutorViewModel.observeSentStudentInterestIds()</code>
tuition_requests	parentId ASC, status ASC	<code>ParentViewModel.getMyPosts()</code>
tuition_requests	status ASC	<code>TutorViewModel.observeAvailableTutions()</code> (single-field, auto-indexed)
referrals	referrerId ASC	<code>ReferralViewModel</code> (single-field, auto-indexed)
tutor_requests	tutorId ASC	<code>TutorViewModel.observeSentTutorRequestIds()</code> (single-field, auto-indexed)

Index notes

- The `applications` collection has the most compound queries. Every new query that combines two `whereEqualTo` clauses will need a composite index. Check the Firebase Console → Firestore → Indexes if a query silently returns empty results.
- `TutorViewModel.observeAppliedRequestIds()` filters client-side by status list (not a Firestore `whereIn`) — no additional index needed beyond `tutorId`.
- `ParentViewModel.observeApplicationStatus()` uses three `whereEqualTo` clauses (`tutorId`, `parentId`, `studentId`) — this requires a three-field composite index.

14. Data Consistency Risks

Risk 1 — Non-atomic multi-document updates

Several state transitions update multiple documents without a Firestore transaction:

Operation	Documents modified	Risk
<code>appointTutor()</code>	Selected app (status= <code>tuition_started</code>), all	If the device goes offline mid-sequence, some apps may

Operation	Documents modified	Risk
	other apps (status=rejected), tuition_requests (status=closed), multiple users (freeDemoEligible)	remain un-rejected and tuition_requests.stat us may stay "open", causing duplicate appointments
acceptDeal() — post source	Application (status=demo_pending_paymen t), all other apps for same request (status=rejected), tuition_requests.acce ptedTutorId	Same partial-write risk
deleteCurrentUser() — parent	users/{uid}, parents/{uid}, all students/{parentId=ui d}	Tuition requests and applications are NOT deleted; orphaned records remain

Recommendation: High-value multi-document transitions should use WriteBatch or Firestore transactions. appointTutor and acceptDeal are the highest priority.

Risk 2 — direct_requests vs applications ambiguity

TutorViewModel.acceptDirectRequest() and rejectDirectRequest() update the direct_requests collection. However, all observed creation paths for direct-flow deals write to applications (not direct_requests). The direct_requests collection may be:

- A legacy collection from an earlier architecture
- A partially implemented parallel flow

No code was found that creates direct_requests documents. If the acceptDirectRequest / rejectDirectRequest methods are called on an ID that doesn't exist in direct_requests, Firestore creates the document with only the status field — a ghost document with no other fields.

Recommendation: Audit whether direct_requests is ever read or queried. If not, the collection can be removed and acceptDirectRequest/rejectDirectRequest should update applications instead.

Risk 3 — nextPaymentDate vs nextDueDate duplication

Application has both nextPaymentDate: String and nextDueDate: String. Both appear to serve the same purpose. EarningsViewModel reads nextDueDate for the earnings dashboard. It is unclear which is canonical.

Recommendation: Decide on one field and remove the other.

Risk 4 — Student deletion does not cascade

Deleting a student via `StudentViewModel.deleteStudent()` only removes the `students/{id}` document. It does not:

- Close the linked `tuition_requests`
- Withdraw linked `applications`
- Update any denormalized `studentName` fields

The runtime name enrichment will silently return an empty name for any application whose student document was deleted.

Risk 5 — `applications.id` field vs document ID

`Application` has an `id: String` body field written explicitly at creation. `Firestore.toObject()` deserializes it from the body, not from the document ID. Old application documents created before this field was added (or via paths that omit it) have `id=""` after deserialization. The `studentNames` map in `EarningsViewModel` and `TutorHomeScreen` is keyed by `app.id`. If `app.id == ""`, all such apps share the same map key — only the last-resolved name is visible.

Status: The forward fix in `applyToTuition` ensures new documents always have `id` in the body. Legacy documents are mitigated by the `requestId` enrichment path which correctly resolves the student name regardless of key collision.

Risk 6 — Negotiation turn-gating is asymmetric

`ParentViewModel.parentCounter()` explicitly checks `lastUpdatedBy != "parent"` before proceeding. `TutorViewModel.tutorCounter()` does not perform the same check. A tutor could theoretically submit multiple consecutive counter-offers without waiting for a parent response, manipulating `currentOffer` freely.

Recommendation: Add the same `lastUpdatedBy != "tutor"` guard to `tutorCounter()`.

Risk 7 — FCM notification fires for direct-flow applications

`notifyParentOnApplication` triggers on ALL new `applications` documents including `source="direct"`. For direct-flow apps, `requestId` is blank. The notification's deep-link data (`route: "applicants", requestId: ""`) will navigate the parent to a broken or empty `applicants` screen.

Recommendation: Add a `source != "direct"` guard in the Cloud Function, or send a

different notification body/route for direct applications.

Risk 8 — No Firestore Security Rules in codebase

No `firestore.rules` file was found in the repository. All Firestore reads and writes may be governed by default rules (deny all, or the Firebase Console's manually set rules). If rules are not enforced:

- Any authenticated user can read all `applications`, `tuition_requests`, and `students` across all parents and tutors
- Tutors can modify parent data and vice versa

Recommendation: Add `firestore.rules` to the repository under source control and enforce role-based access per collection.

Appendix: UI-only models (not persisted to Firestore)

Model	Purpose
TutorProfile	Lightweight subset of <code>Tutor</code> used in display cards and the recommendation list. Mapped client-side from <code>Tutor</code> objects or individual document reads.
TuitionUI	UI rendering wrapper (title, subtitle, distance, price). Assembled from <code>TuitionRequest</code> data in the UI layer. Never written to Firestore.
StudentFormOptions	Static option lists for student forms (boards, subjects, etc.). Compile-time constants, not database.
TutorFormOptions	Static option lists for tutor profile forms. Compile-time constants.